



**Universidad Nacional
Autónoma de México**
Facultad de Ingeniería



Diseño Digital VLSI

Proyecto Final

**Robot móvil seguidor de luz con evasión de
obstáculos**

Integrantes:

Almodovar Tufiño Tarek - 423127375

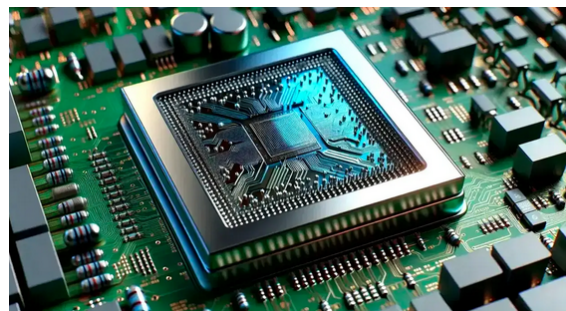
Najera Higuera Junior Josue - 421112869

Romero Calixto Carlo Magno - 320187890

Lopez Gonzalez Hector Albino - 320342732

Fecha de entrega:

05 de Junio de 2026



Índice

1. Objetivo	3
2. Introducción	3
3. Especificaciones del proyecto	3
4. Arquitectura general del sistema	4
5. Diagrama de bloques	4
6. Asignación de pines	5
7. Máquina de Estados Finitos	6
8. Funcionamiento del código	6
8.1. Módulo principal: top.vhd	6
8.2. Módulo ultrasónico: ultrasonico_hcsr04.vhd	7
8.3. FSM principal: fsm.vhd	7
9. Desarrollo e implementación	8
9.1. Construcción física del robot	8
9.2. Sensores de luz	9
9.3. Sensor ultrasónico	9
9.4. Driver de motores	10
9.5. Estructura de detección de luz	11
10. Recursos utilizados	11
10.1. Recursos de hardware	11
10.2. Recursos de software	12
11. Verificación y pruebas	12
11.1. Prueba de sensores LDR	12
11.2. Prueba del sensor ultrasónico	12
11.3. Prueba de motores	13
11.4. Prueba de la FSM	13
11.5. Prueba de integración	13
12. Resultados obtenidos	13
13. Video funcional	14
14. Repositorio del proyecto	14
15. Límites del proyecto	14

16.Mejoras futuras	14
17.Conclusiones	15
18.Referencias	15
19.Anexos	16
19.1. Código del módulo principal	16
19.2. Código de la Máquina de Estados Finitos	19
19.3. Código del módulo ultrasónico HC-SR04	26

1. Objetivo

Diseñar e implementar un robot móvil capaz de alcanzar una fuente luminosa evitando obstáculos de manera autónoma y robusta.

El propósito del proyecto es integrar los conocimientos vistos durante el curso de Diseño Digital VLSI mediante el desarrollo de un sistema físico controlado por lógica digital. Para ello, se emplea una tarjeta FPGA Intel DE10-Lite, sensores de luz, un sensor ultrasónico frontal, un driver de motores y una Máquina de Estados Finitos descrita en VHDL.

2. Introducción

El presente proyecto consiste en el diseño e implementación de un robot móvil seguidor de luz con evasión de obstáculos. La idea principal es que el robot pueda detectar una fuente luminosa, desplazarse hacia ella y evitar un obstáculo cuando este se encuentre cerca de su trayectoria.

Para lograrlo, se utilizaron sensores de luz tipo LDR, los cuales permiten identificar la dirección de mayor intensidad luminosa. Además, se integró un sensor ultrasónico HC-SR04 para detectar obstáculos al frente del robot. La información proveniente de los sensores es procesada por una Máquina de Estados Finitos implementada en VHDL, la cual decide el comportamiento del sistema.

El robot fue construido sobre una base móvil con dos motores de corriente directa. Estos motores son controlados mediante un driver de potencia, mientras que la FPGA se encarga de generar las señales lógicas de dirección. De esta manera, el proyecto permite observar cómo un diseño digital puede interactuar con componentes físicos reales.

La importancia de este trabajo se encuentra en la integración de diferentes áreas: electrónica digital, sensores, actuadores, lógica secuencial, máquinas de estados, temporizadores y programación en VHDL. Además, el proyecto permitió comprobar la relación entre el diseño teórico y la implementación práctica en hardware.

3. Especificaciones del proyecto

Las especificaciones consideradas para el diseño del robot fueron las siguientes:

- Control de dos motores de corriente directa mediante un driver de potencia.
- Uso de sensores de luz para determinar la dirección de la fuente luminosa.
- Uso de sensor ultrasónico para detección de obstáculos.
- Implementación de una Máquina de Estados Finitos principal.
- Visualización del estado del robot mediante LEDs.
- Integración del sistema en una tarjeta Intel DE10-Lite.
- Prueba funcional en una trayectoria con fuente luminosa y obstáculo.

En la implementación final se trabajó con un sensor ultrasónico frontal. Aunque la especificación original proponía el uso de varios sensores de distancia, en esta versión se priorizó una implementación funcional y estable con detección frontal de obstáculos.

4. Arquitectura general del sistema

La arquitectura del sistema se diseñó de manera modular para facilitar el desarrollo, las pruebas y la integración del robot. Cada módulo cumple una función específica y se comunica con los demás mediante señales digitales.

Los bloques principales del sistema son:

- **Sensores LDR:** detectan la presencia de luz en diferentes posiciones del robot.
- **Sensor ultrasónico HC-SR04:** mide la distancia frontal hacia un posible obstáculo.
- **Módulo ultrasónico en VHDL:** genera la señal *trigger*, mide el pulso *echo* y determina si existe un obstáculo a 20 cm o menos.
- **Máquina de Estados Finitos:** controla el comportamiento general del robot.
- **Driver de motores:** recibe las señales lógicas de la FPGA y entrega potencia a los motores.
- **Motores DC:** permiten el desplazamiento físico del robot.
- **LEDs de estado:** indican el estado actual de la FSM.

El archivo `top.vhd` funciona como módulo principal del proyecto. En él se conectan el módulo ultrasónico `ultrasonico_hcsr04.vhd` y la FSM principal `fsm.vhd`.

5. Diagrama de bloques

El siguiente diagrama representa la arquitectura general del sistema implementado.

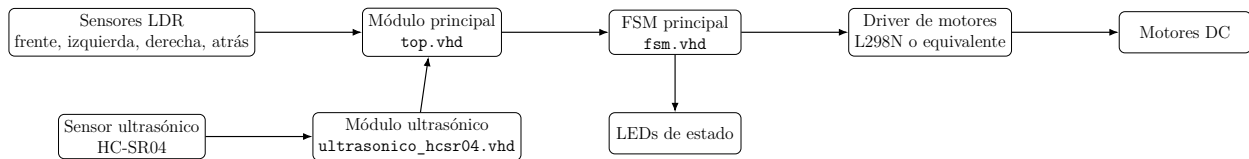


Figura 1: Diagrama de bloques general del robot móvil.

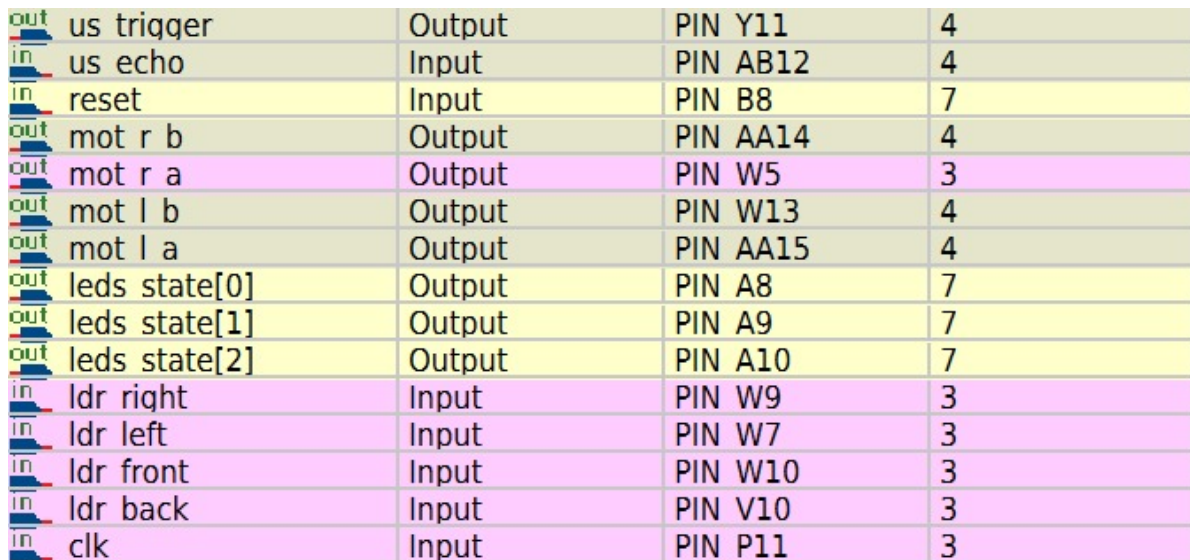
El bloque central del sistema es la FSM, ya que toma decisiones con base en las entradas de luz y distancia. Dependiendo de las condiciones detectadas, el robot puede permanecer detenido, seguir la luz, esquivar un obstáculo o detenerse al llegar a la meta.

6. Asignación de pines

La asignación de pines se realizó en el Pin Planner de Quartus. La siguiente tabla muestra las señales principales utilizadas en la implementación.

Cuadro 1: Asignación de pines utilizada en la tarjeta DE10-Lite.

Señal	Dirección	Pin
clk	Entrada	PIN_P11
reset	Entrada	PIN_B8
ldr_front	Entrada	PIN_W10
ldr_left	Entrada	PIN_V10
ldr_right	Entrada	PIN_W9
ldr_back	Entrada	PIN_V9
us_echo	Entrada	PIN_AB12
us_trigger	Salida	PIN_Y11
mot_l_a	Salida	PIN_AA15
mot_l_b	Salida	PIN_W13
mot_r_a	Salida	PIN_W5
mot_r_b	Salida	PIN_AA14
leds_state[0]	Salida	PIN_A8
leds_state[1]	Salida	PIN_A9
leds_state[2]	Salida	PIN_A10



out	us trigger	Output	PIN Y11	4
in	us echo	Input	PIN AB12	4
in	reset	Input	PIN B8	7
out	mot r b	Output	PIN AA14	4
out	mot r a	Output	PIN W5	3
out	mot l b	Output	PIN W13	4
out	mot l a	Output	PIN AA15	4
out	leds state[0]	Output	PIN A8	7
out	leds state[1]	Output	PIN A9	7
out	leds state[2]	Output	PIN A10	7
in	ldr right	Input	PIN W9	3
in	ldr left	Input	PIN W7	3
in	ldr front	Input	PIN W10	3
in	ldr back	Input	PIN V10	3
in	clk	Input	PIN P11	3

Figura 2: Captura del Pin Planner con la asignación de entradas y salidas del proyecto.

7. Máquina de Estados Finitos

La lógica principal del robot se implementó mediante una Máquina de Estados Finitos. Esta estructura permite dividir el comportamiento del robot en estados definidos y controlar las transiciones de forma ordenada.

Los estados implementados en el archivo `fsm.vhd` son:

- **ESTADO_INICIO:** estado inicial. El robot permanece detenido durante aproximadamente un segundo antes de comenzar.
- **ESTADO_BUSCAR_LUZ:** el robot permanece detenido esperando detectar luz en alguno de los sensores LDR.
- **ESTADO_SEGUIR_LUZ:** el robot activa los motores según la dirección donde se detecta la luz.
- **ESTADO_ESQUIVE:** el robot entra a este estado cuando el sensor ultrasónico detecta un obstáculo a 20 cm o menos.
- **ESTADO_META:** el robot se detiene cuando se cumple la condición de llegada a la fuente luminosa.

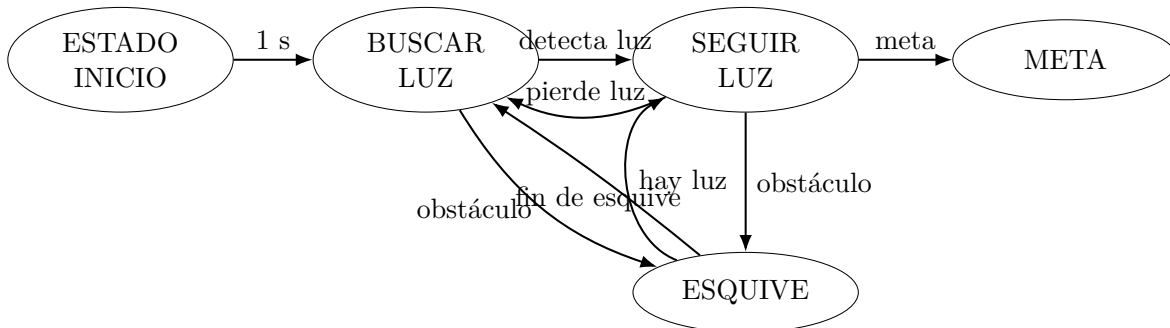


Figura 3: Máquina de Estados Finitos implementada en el robot.

La FSM da prioridad a la detección de obstáculos. Si el robot se encuentra en búsqueda o seguimiento de luz y el sensor ultrasónico detecta un objeto cercano, el sistema cambia automáticamente al estado de esquivar.

8. Funcionamiento del código

El proyecto se divide en tres archivos principales: `top.vhd`, `fsm.vhd` y `ultrasonico_hcsr04.vhd`.

8.1. Módulo principal: `top.vhd`

El archivo `top.vhd` funciona como módulo de mayor jerarquía. En este archivo se declaran las entradas y salidas físicas de la tarjeta FPGA y se instancian los módulos internos.

Sus funciones principales son:

- Recibir las entradas de los sensores LDR.
- Recibir la señal `echo` del sensor ultrasónico.
- Generar la salida `us_trigger`.
- Conectar la señal de obstáculo con la FSM.
- Enviar las señales de control hacia el driver de motores.
- Mostrar el estado actual mediante LEDs.

8.2. Módulo ultrasónico: `ultrasonico_hcsr04.vhd`

El archivo `ultrasonico_hcsr04.vhd` se encarga de controlar el sensor HC-SR04. Su funcionamiento se basa en generar un pulso de activación y medir el tiempo que permanece activa la señal de eco.

En el código se genera un pulso `trigger` de 10 microsegundos. Después, el módulo espera la señal `echo` y cuenta el tiempo que esta permanece en alto. Si el tiempo medido corresponde a una distancia menor o igual a 20 cm, se activa la señal `obstaculo_20cm`.

El módulo usa los siguientes parámetros principales:

- Frecuencia de reloj: 50 MHz.
- Pulso de trigger: 10 μ s.
- Periodo de medición: 60 ms.
- Tiempo máximo de espera: 30 ms.
- Distancia límite: 20 cm.

8.3. FSM principal: `fsm.vhd`

El archivo `fsm.vhd` contiene la lógica de control del robot. En este módulo se procesan las señales de luz y distancia para decidir el movimiento de los motores.

Una parte importante del código es la normalización de los sensores LDR. En la implementación, los sensores se consideran activos en bajo, por lo que se define:

```
constant LDR_ACTIVO : STD_LOGIC := '0';
```

También se implementa un pequeño filtro de estabilidad o *debounce* para evitar cambios rápidos causados por ruido o variaciones pequeñas en la lectura de los sensores. Este filtro ayuda a que la FSM no cambie de estado por lecturas momentáneas.

Durante el estado de esquite, el robot activa únicamente el motor izquierdo hacia adelante durante un tiempo aproximado de 650 ms. Esto provoca que el robot cambie su orientación para evitar el obstáculo detectado al frente.

9. Desarrollo e implementación

El desarrollo del proyecto se realizó por etapas. Primero se revisaron los requerimientos del robot y se definieron los componentes necesarios para cumplir con la función principal. Después se realizó la conexión de sensores, motores y tarjeta FPGA.

9.1. Construcción física del robot

El robot fue construido sobre una base sencilla, utilizando materiales ligeros para sostener la tarjeta DE10-Lite, los sensores, el driver de motores y la alimentación. La estructura se armó con una combinación de base plástica, cartón, madera y soportes para colocar los sensores en diferentes posiciones.

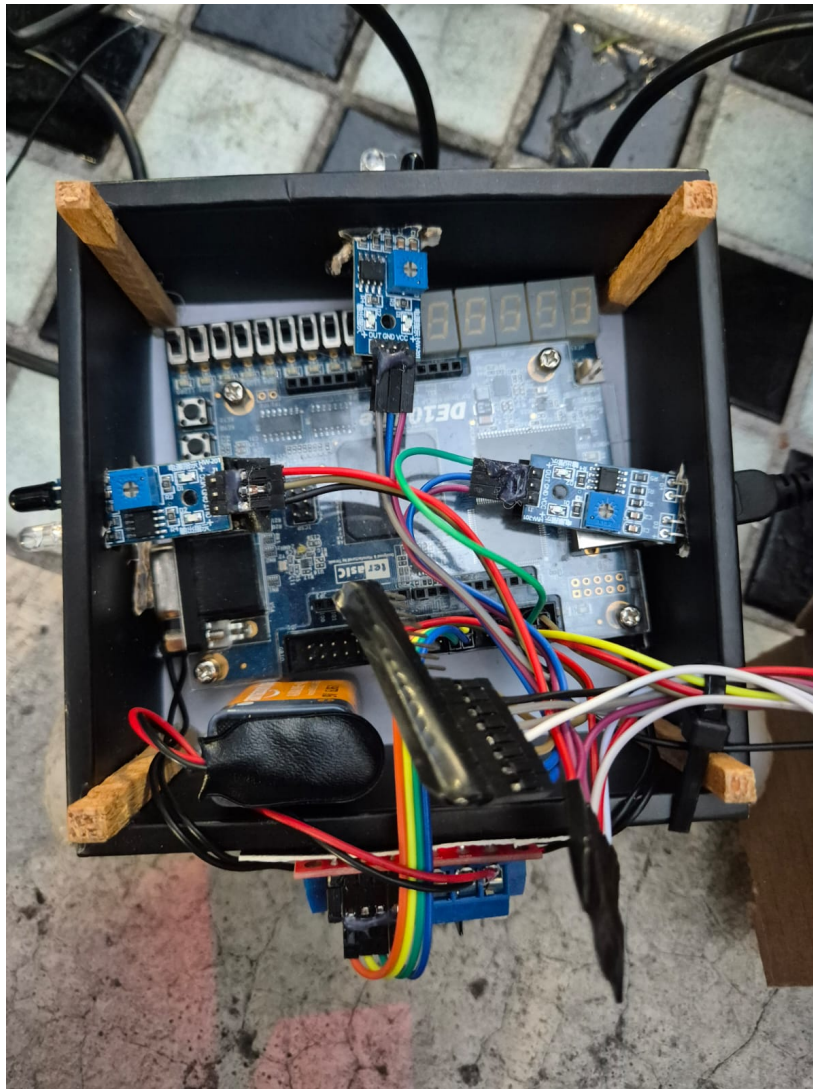


Figura 4: Vista superior del robot con la tarjeta DE10-Lite y sensores integrados.

9.2. Sensores de luz

Para detectar la fuente luminosa se utilizaron sensores LDR montados en diferentes partes de la estructura. Estos sensores permiten identificar de qué dirección proviene la luz. La salida digital de cada módulo se conecta directamente a la FPGA.

Los sensores LDR utilizados cuentan con un potenciómetro de ajuste, lo que permite calibrar la sensibilidad dependiendo de la luz ambiente. Esta calibración fue importante porque el robot debía distinguir entre la iluminación normal del entorno y la fuente luminosa objetivo.

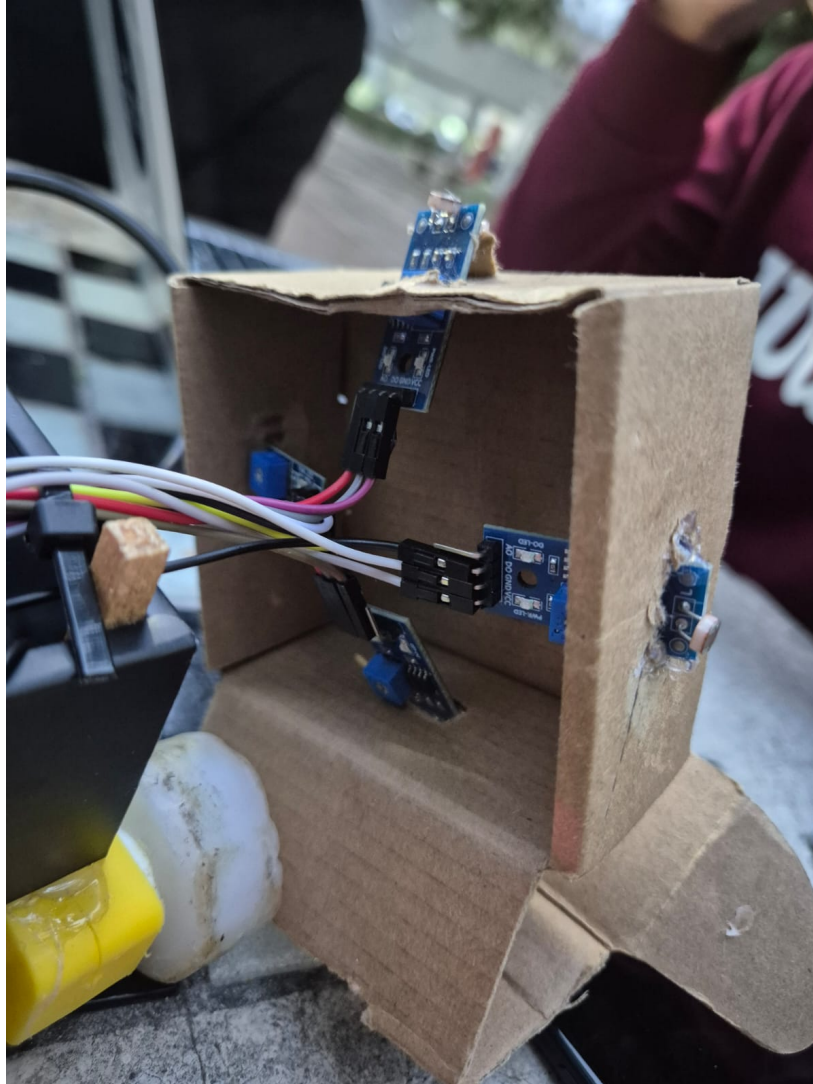


Figura 5: Ubicación de sensores LDR en la estructura de detección de luz.

9.3. Sensor ultrasónico

Para la detección de obstáculos se utilizó un sensor ultrasónico HC-SR04. Este sensor se colocó en la parte frontal del robot para detectar objetos que pudieran bloquear su trayectoria.

El sensor ultrasónico trabaja mediante una señal de disparo y una señal de eco. La FPGA genera la señal de disparo y después mide cuánto tiempo tarda en regresar el eco. Con base en ese tiempo se determina si el obstáculo se encuentra dentro del rango de riesgo.

9.4. Driver de motores

El control de potencia de los motores se realizó mediante un driver de motores. Este componente recibe señales lógicas desde la FPGA y permite alimentar los motores con la corriente necesaria para su funcionamiento.

En la implementación final, los pines de habilitación del driver se mantuvieron activos mediante jumper a VCC. Por esta razón, el control principal se hizo mediante las señales de dirección de los motores, no mediante PWM.

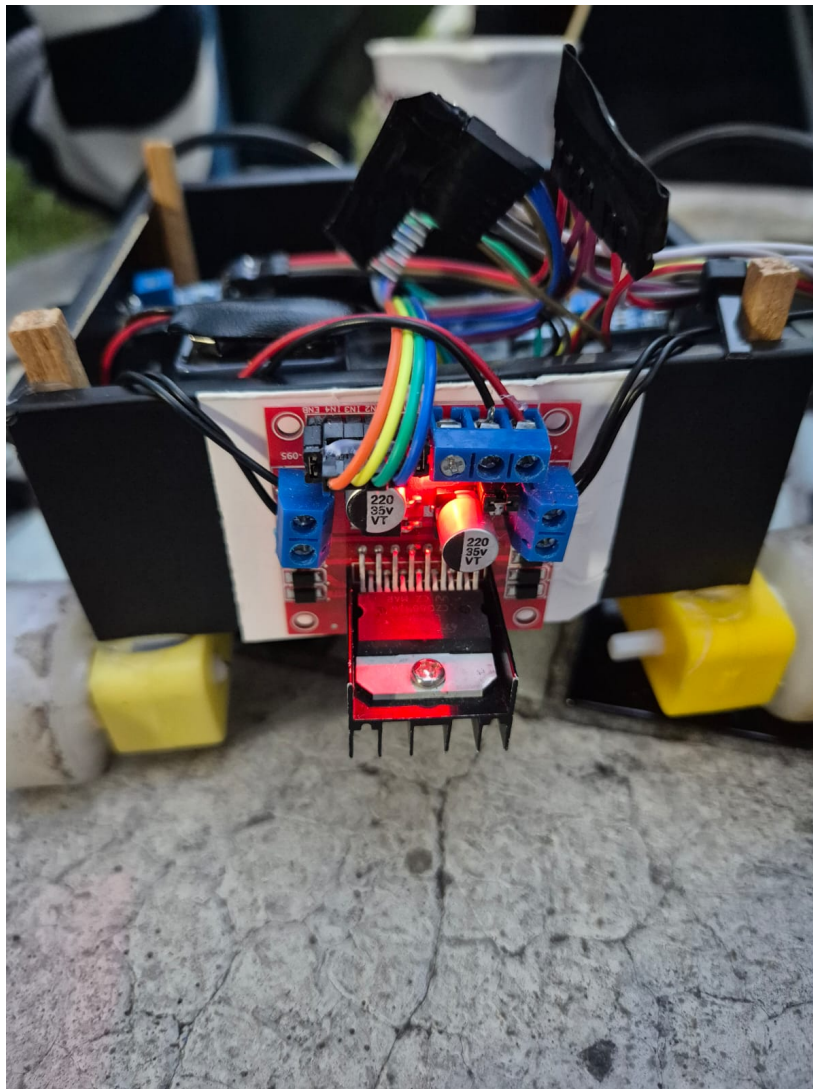


Figura 6: Driver de motores conectado a los motores DC del robot.

9.5. Estructura de detección de luz

Se construyó una estructura adicional para colocar los sensores LDR y reducir interferencias entre ellos. Esta estructura ayuda a que cada sensor tenga una orientación diferente y pueda detectar con mayor claridad la dirección de la fuente luminosa.



Figura 7: Estructura utilizada para orientar los sensores de luz.

10. Recursos utilizados

10.1. Recursos de hardware

- Tarjeta FPGA Intel DE10-Lite.
- Dos motores de corriente directa.
- Driver de motores tipo L298N o equivalente.

- Sensores LDR con salida digital.
- Sensor ultrasónico HC-SR04.
- Cables Dupont.
- Fuente de alimentación o batería.
- Base del robot.
- Materiales de soporte como cartón, madera y silicón.
- LEDs de la tarjeta para visualizar estados.

10.2. Recursos de software

- Lenguaje de descripción de hardware VHDL.
- Intel Quartus Prime.
- Pin Planner para asignación de pines.
- Editor de código.
- Herramientas para elaboración de diagramas.

11. Verificación y pruebas

La verificación del proyecto se realizó mediante pruebas por módulos y posteriormente mediante pruebas de integración.

11.1. Prueba de sensores LDR

Se probaron los sensores LDR colocando una fuente luminosa en diferentes posiciones alrededor del robot. El objetivo fue verificar que las señales digitales cambiaran correctamente al recibir luz.

Durante esta prueba se ajustaron los potenciómetros de los módulos LDR para definir una sensibilidad adecuada. Esto permitió reducir falsas detecciones provocadas por la luz ambiente.

11.2. Prueba del sensor ultrasónico

El sensor ultrasónico se probó colocando obstáculos a diferentes distancias frente al robot. La distancia límite utilizada en el código fue de 20 cm. Cuando el obstáculo se encontraba dentro de ese rango, la señal `obstaculo_20cm` se activaba.

Esta prueba fue importante porque la FSM depende de dicha señal para entrar al estado de esquivar.

11.3. Prueba de motores

Se probaron las señales de dirección enviadas al driver de motores. Con estas señales se verificaron los movimientos básicos del robot, como avance, giro y paro.

Debido a que los pines de habilitación del driver permanecen activos, los motores funcionan a velocidad fija y el cambio de movimiento se realiza modificando las señales de dirección.

11.4. Prueba de la FSM

Se verificaron los cambios de estado utilizando los LEDs de la tarjeta. Cada estado de la FSM se representa mediante la salida `leds_state`, lo que facilitó identificar el comportamiento del sistema durante la prueba.

La codificación de estados fue la siguiente:

Cuadro 2: Codificación de estados mediante LEDs.

Estado	LEDs
ESTADO_INICIO	000
ESTADO_BUSCAR_LUZ	001
ESTADO_SEGUIR_LUZ	010
ESTADO_ESQUIVE	011
ESTADO_META	111

11.5. Prueba de integración

Finalmente se probó el robot completo, integrando sensores, FPGA, driver de motores y estructura física. En esta prueba se verificó que el robot pudiera detectar luz, seguirla y ejecutar una maniobra de esquivar cuando el sensor ultrasónico detectaba un obstáculo frontal.

12. Resultados obtenidos

Durante las pruebas realizadas se logró integrar el sistema principal del robot. El robot pudo leer señales provenientes de sensores LDR, detectar obstáculos mediante el sensor ultrasónico y modificar el comportamiento de sus motores de acuerdo con la lógica programada en VHDL.

La FSM permitió organizar el comportamiento del robot de forma clara. El estado de inicio dio un pequeño retardo antes de comenzar, el estado de búsqueda permitió esperar una señal de luz, el estado de seguimiento activó los motores según la dirección detectada, el estado de esquivar permitió evitar obstáculos y el estado de meta detuvo el robot cuando se cumplió la condición establecida.

El sensor ultrasónico funcionó como mecanismo de protección frontal. Cuando se detectaba un obstáculo a 20 cm o menos, el robot interrumpía el seguimiento de luz y entraba al estado de esquivar. Esta prioridad ayudó a evitar choques durante la prueba.

Como resultado general, se obtuvo un prototipo funcional que combina lectura de sensores, control secuencial, señales hacia motores y visualización de estados en una FPGA.

13. Video funcional

Como evidencia del funcionamiento del proyecto, se realizó un video demostrativo en el que se observa el comportamiento del robot. En el video se muestra la detección de luz, el movimiento del robot y la respuesta ante obstáculos.

Enlace al video: <https://goo.su/L8QA6Ju>

14. Repositorio del proyecto

El código fuente, archivos de diseño y documentación complementaria del proyecto se encuentran disponibles en el siguiente repositorio:

Enlace al repositorio: <https://github.com/junior-programeer/ProyectoRobot.git>

15. Límites del proyecto

Durante el desarrollo del proyecto se identificaron algunas limitaciones importantes.

La primera limitación fue la sensibilidad de los sensores LDR. Estos sensores dependen de la iluminación del entorno, por lo que fue necesario calibrarlos antes de realizar las pruebas. Si la luz ambiente era muy intensa, podía afectar la detección de la fuente luminosa.

Otra limitación fue el uso de un solo sensor ultrasónico frontal. Esto permite detectar obstáculos al frente, pero no ofrece una lectura completa del entorno lateral o trasero. Por lo tanto, el robot puede esquivar obstáculos frontales, pero no cuenta con una detección completa en todas las direcciones.

También se identificó que el control de velocidad no se implementó mediante PWM en la versión final. Los pines de habilitación del driver se mantuvieron activos, por lo que el movimiento se controló principalmente mediante las señales de dirección. Como mejora futura, se podría integrar PWM para controlar la velocidad de cada motor.

Además, la estructura física del robot fue construida con materiales ligeros y de fácil acceso, lo cual permitió realizar pruebas rápidas, pero también puede afectar la estabilidad mecánica del sistema. Una estructura más rígida ayudaría a mejorar la repetibilidad de las pruebas.

Finalmente, la precisión del comportamiento depende de la correcta calibración de sensores, la carga de la batería y el estado mecánico de los motores. Si un motor gira con mayor fuerza que el otro, el robot puede desviarse de su trayectoria.

16. Mejoras futuras

Como mejoras futuras se proponen las siguientes:

- Agregar más sensores ultrasónicos para detectar obstáculos al frente, izquierda, derecha y parte trasera.
- Implementar PWM para controlar la velocidad de cada motor.
- Integrar encoders para medir la velocidad real de las ruedas.
- Implementar un control PI o PID para mejorar el desplazamiento.
- Mejorar la estructura física del robot con materiales más rígidos.
- Agregar una interfaz de depuración mediante VGA, UART o displays.
- Optimizar la FSM para realizar maniobras de escape más completas.

17. Conclusiones

- Se logró construir un robot móvil funcional capaz de detectar una fuente luminosa y modificar su movimiento con base en las señales de sensores LDR.
- Se implementó una Máquina de Estados Finitos en VHDL para controlar el comportamiento general del robot. Esta FSM permitió organizar el sistema en estados claros como inicio, búsqueda de luz, seguimiento, esquivar y meta.
- Se integró un sensor ultrasónico HC-SR04 para detectar obstáculos frontales. Con esta información, el robot pudo cambiar al estado de esquivar cuando un objeto se encontraba a una distancia menor o igual a 20 cm.
- El uso de la tarjeta Intel DE10-Lite permitió implementar la lógica digital del proyecto en hardware real, conectando sensores y motores con una descripción en VHDL.
- La visualización mediante LEDs facilitó la depuración del sistema, ya que permitió observar el estado actual de la FSM durante las pruebas.
- El proyecto permitió reforzar temas vistos en la materia, como diseño modular, lógica secuencial, temporización, máquinas de estados, asignación de pines e integración de hardware.
- Aunque el prototipo cumple con la función principal, se identificaron áreas de mejora como la incorporación de PWM, más sensores de distancia y una estructura mecánica más robusta.

18. Referencias

- Material proporcionado en clase: *Robot móvil seguidor de luz con evasión de obstáculos*, Diseño Digital VLSI, Proyecto Final, 2026.
- Intel. *DE10-Lite User Manual*.

- Datasheet del sensor ultrasónico HC-SR04.
- Datasheet del driver de motores L298N o equivalente.
- Documentación de Intel Quartus Prime.
- Apuntes y material del curso de Diseño Digital VLSI.

19. Anexos

19.1. Código del módulo principal

```

1  -- =====
2  -- TOP LEVEL - Robot seguidor de luz con esquivе ultrasonico
3  -- FPGA: Intel DE10-Lite (MAX10, 50 MHz)
4  -- Reset: activo bajo (KEY0)
5  -- Sin PWM: Enable A y B con jumper a VCC
6  --
7  -- LDR segun pin planner real:
8  --   ldr_front -> PIN_W10
9  --   ldr_left  -> PIN_V10
10 --   ldr_right -> PIN_W9
11 --   ldr_back  -> PIN_V9
12 --
13 -- Ultrasonico tipo HC-SR04:
14 --   us_trigger -> asignar en Pin Planner
15 --   us_echo    -> asignar en Pin Planner, usando divisor 5V a 3.3V
16 -- =====
17 library IEEE;
18 use IEEE.STD_LOGIC_1164.ALL;
19 use IEEE.NUMERIC_STD.ALL;
20
21 entity top is
22   Port (
23     clk          : in  STD_LOGIC;           -- PIN_P11, 50 MHz
24     reset        : in  STD_LOGIC;           -- PIN_B8, KEY0
25     activo bajo
26
27     ldr_front    : in  STD_LOGIC;           -- PIN_W10
28     ldr_left     : in  STD_LOGIC;           -- PIN_V10
29     ldr_right    : in  STD_LOGIC;           -- PIN_W9
30     ldr_back     : in  STD_LOGIC;           -- PIN_V9
31
32     -- Sensor ultrasonico frontal
33     us_echo      : in  STD_LOGIC;           -- ECHO del sensor,
34     usar divisor a 3.3V
35     us_trigger   : out STD_LOGIC;           -- TRIGGER del
36     sensor
37
38     -- Driver de motores tipo L293D / L298N
39     mot_l_a      : out STD_LOGIC;           -- PIN_AA15 IN1
40     mot_l_b      : out STD_LOGIC;           -- PIN_W13  IN2

```

```

38         mot_r_a      : out STD_LOGIC;           -- PIN_W5    IN3
39         mot_r_b      : out STD_LOGIC;           -- PIN_AA14 IN4
40
41         leds_state   : out STD_LOGIC_VECTOR(2 downto 0) -- PIN_A8/A9/A10
42     );
43 end top;
44
45 architecture Structural of top is
46
47     signal obstaculo_20cm_s : STD_LOGIC;
48     signal medida_lista_s   : STD_LOGIC;
49
50 begin
51
52     -- Modulo del sensor ultrasonico.
53     -- Entrega obstaculo_20cm_s = '1' cuando detecta objeto a 20 cm o
54     -- menos.
55     inst_ultrasonico : entity work.ultrasonico_hcsr04
56     generic map (
57         CLK_FREQ_HZ      => 50_000_000,
58         DIST_THRESHOLD_CM => 20,
59         MEASURE_PERIOD_MS => 60,
60         ECHO_TIMEOUT_MS  => 30
61     )
62     port map (
63         clk           => clk,
64         reset         => reset,
65         echo          => us_echo,
66         trigger       => us_trigger,
67         obstaculo_20cm => obstaculo_20cm_s,
68         medida_lista  => medida_lista_s
69     );
70
71     -- FSM principal: seguidor de luz + estado ESTADO_ESQUIVE.
72     inst_fsm : entity work.fsm
73     generic map (
74         CLK_FREQ_HZ      => 50_000_000,
75         ESQUIVE_TIME_MS  => 650
76     )
77     port map (
78         clk           => clk,
79         reset         => reset,
80
81         ldr_front     => ldr_front,
82         ldr_left      => ldr_left,
83         ldr_right     => ldr_right,
84         ldr_back      => ldr_back,
85
86         obstaculo_20cm => obstaculo_20cm_s,
87
88         mot_l_a       => mot_l_a,
89         mot_l_b       => mot_l_b,
90         mot_r_a       => mot_r_a,
91         mot_r_b       => mot_r_b,

```

```
91  
92         leds_state      => leds_state  
93     );  
94  
95 end Structural;
```

Listing 1: Código del módulo principal top.vhd

19.2. Código de la Máquina de Estados Finitos

```
1  -- =====
2  -- FSM - Robot seguidor de luz con estado de esquivar
3  -- FPGA: Intel DE10-Lite (MAX10, 50 MHz)
4  -- Reset: activo bajo (KEY0)
5  --
6  -- Estados:
7  --     ESTADO_INICIO      : 1 segundo parado
8  --     ESTADO_BUSCAR_LUZ  : detenido hasta detectar luz
9  --     ESTADO_SEGUIR_LUZ  : mueve motores segun LDR
10 --     ESTADO_ESQUIVE     : inicia con obstaculo a 20 cm o menos
11 --     ESTADO_META        : parado al cumplir condicion de meta
12 --
13 -- Motor:
14 --     PARADO      : mot_X_a='0', mot_X_b='0'
15 --     ADELANTE    : mot_X_a='1', mot_X_b='0'
16 --     ATRAS       : mot_X_a='0', mot_X_b='1'
17 -- =====
18 library IEEE;
19 use IEEE.STD_LOGIC_1164.ALL;
20 use IEEE.NUMERIC_STD.ALL;
21
22 entity fsm is
23     generic (
24         CLK_FREQ_HZ      : positive := 50_000_000;
25         ESQUIVE_TIME_MS  : positive := 650
26     );
27     Port (
28         clk              : in  STD_LOGIC;
29         reset            : in  STD_LOGIC;
30
31         ldr_front        : in  STD_LOGIC;    -- PIN_W10
32         ldr_left         : in  STD_LOGIC;    -- PIN_V10
33         ldr_right        : in  STD_LOGIC;    -- PIN_W9
34         ldr_back         : in  STD_LOGIC;    -- PIN_V9
35
36         obstaculo_20cm   : in  STD_LOGIC;    -- '1' si hay objeto a 20 cm o
            menos
37
38         mot_l_a          : out STD_LOGIC;    -- PIN_AA15 IN1
39         mot_l_b          : out STD_LOGIC;    -- PIN_W13  IN2
40         mot_r_a          : out STD_LOGIC;    -- PIN_W5   IN3
41         mot_r_b          : out STD_LOGIC;    -- PIN_AA14 IN4
42
43         leds_state       : out STD_LOGIC_VECTOR(2 downto 0)
44     );
45 end fsm;
46
47 architecture Behavioral of fsm is
48
49     constant LDR_ACTIVO : STD_LOGIC := '0';
50
51     type state_type is (
```

```

52     ESTADO_INICIO,
53     ESTADO_BUSCAR_LUZ,
54     ESTADO_SEGUIR_LUZ,
55     ESTADO_ESQUIVE,
56     ESTADO_META
57 );
58
59 signal estado : state_type := ESTADO_INICIO;
60
61 constant ARRANQUE_MAX : integer := 50_000_000;
62 signal arranque_cnt : integer range 0 to 50_000_000 := 0;
63
64 constant DB_MAX : integer := 2_500_000;
65 signal db_f : integer range 0 to 2_500_000 := 0;
66 signal db_l : integer range 0 to 2_500_000 := 0;
67 signal db_r : integer range 0 to 2_500_000 := 0;
68 signal db_b : integer range 0 to 2_500_000 := 0;
69
70 signal luz_front : STD_LOGIC := '0';
71 signal luz_left : STD_LOGIC := '0';
72 signal luz_right : STD_LOGIC := '0';
73 signal luz_back : STD_LOGIC := '0';
74
75 function ms_to_ticks(ms_value : positive) return natural is
76 begin
77     return (CLK_FREQ_HZ / 1_000) * ms_value;
78 end function;
79
80 constant ESQUIVE_MAX : natural := ms_to_ticks(ESQUIVE_TIME_MS);
81 signal esquivе_cnt : natural range 0 to ESQUIVE_MAX := 0;
82
83 begin
84
85     -- =====
86     -- DEBOUNCE + NORMALIZACION LDR
87     -- =====
88     process(clk, reset)
89     begin
90         if reset = '0' then
91             db_f <= 0; luz_front <= '0';
92             db_l <= 0; luz_left <= '0';
93             db_r <= 0; luz_right <= '0';
94             db_b <= 0; luz_back <= '0';
95
96             elsif rising_edge(clk) then
97
98                 if ldr_front = LDR_ACTIVO then
99                     if db_f < DB_MAX then
100                         db_f <= db_f + 1;
101                     else
102                         luz_front <= '1';
103                     end if;
104                 else
105                     db_f <= 0;

```

```

106         luz_front <= '0';
107     end if;
108
109     if ldr_left = LDR_ACTIVO then
110         if db_l < DB_MAX then
111             db_l <= db_l + 1;
112         else
113             luz_left <= '1';
114         end if;
115     else
116         db_l <= 0;
117         luz_left <= '0';
118     end if;
119
120     if ldr_right = LDR_ACTIVO then
121         if db_r < DB_MAX then
122             db_r <= db_r + 1;
123         else
124             luz_right <= '1';
125         end if;
126     else
127         db_r <= 0;
128         luz_right <= '0';
129     end if;
130
131     if ldr_back = LDR_ACTIVO then
132         if db_b < DB_MAX then
133             db_b <= db_b + 1;
134         else
135             luz_back <= '1';
136         end if;
137     else
138         db_b <= 0;
139         luz_back <= '0';
140     end if;
141
142     end if;
143 end process;
144
145 -- =====
146 -- FSM PRINCIPAL
147 --
148 -- Prioridad de obstaculo:
149 -- Si obstaculo_20cm = '1' durante BUSCAR_LUZ o SEGUIR_LUZ,
150 -- entra a ESTADO_ESQUIVE.
151 --
152 -- ESTADO_ESQUIVE:
153 -- Solo activa un motor para girar hacia un lado.
154 -- Aqui se activa solo el motor izquierdo hacia adelante.
155 -- Para girar al lado contrario, apaga el izquierdo y activa el
156 -- derecho.
157 -- =====
158 process(clk, reset)
159 begin

```

```

159     if reset = '0' then
160         estado      <= ESTADO_INICIO;
161         arranque_cnt <= 0;
162         esquite_cnt <= 0;
163         mot_l_a      <= '0';
164         mot_l_b      <= '0';
165         mot_r_a      <= '0';
166         mot_r_b      <= '0';
167         leds_state   <= "000";
168
169     elsif rising_edge(clk) then
170
171         -- Prioridad del ultrasonico: entra a ESQUIRE.
172         if (estado = ESTADO_BUSCAR_LUZ or estado = ESTADO_SEGUIR_LUZ)
173             and
174             (obstaculo_20cm = '1') then
175
176             estado      <= ESTADO_ESQUIRE;
177             esquite_cnt <= 0;
178
179             -- Un solo motor activo: motor izquierdo adelante.
180             mot_l_a      <= '1';
181             mot_l_b      <= '0';
182             mot_r_a      <= '0';
183             mot_r_b      <= '0';
184             leds_state   <= "011";
185
186         else
187             case estado is
188
189                 -- =====
190                 -- INICIO: 1 segundo parado antes de arrancar
191                 -- =====
192                 when ESTADO_INICIO =>
193                     leds_state <= "000";
194                     mot_l_a     <= '0';
195                     mot_l_b     <= '0';
196                     mot_r_a     <= '0';
197                     mot_r_b     <= '0';
198                     esquite_cnt <= 0;
199
200                     if arranque_cnt = ARRANQUE_MAX then
201                         arranque_cnt <= 0;
202                         estado      <= ESTADO_BUSCAR_LUZ;
203                     else
204                         arranque_cnt <= arranque_cnt + 1;
205                     end if;
206
207                 -- =====
208                 -- BUSCAR_LUZ: robot detenido mientras espera luz
209                 -- =====
210                 when ESTADO_BUSCAR_LUZ =>
211                     leds_state <= "001";
212                     esquite_cnt <= 0;

```

```

212
213     mot_l_a <= '0';
214     mot_l_b <= '0';
215     mot_r_a <= '0';
216     mot_r_b <= '0';
217
218     if luz_front = '1' or luz_left = '1' or
219        luz_right = '1' or luz_back = '1' then
220         estado <= ESTADO_SEGUIR_LUZ;
221     end if;
222
223     -- =====
224     -- SEGUIR_LUZ
225     -- =====
226     when ESTADO_SEGUIR_LUZ =>
227         leds_state <= "010";
228         esquive_cnt <= 0;
229
230         if luz_front = '1' and luz_left = '1' and
231            luz_right = '1' then
232             estado <= ESTADO_META;
233             mot_l_a <= '0'; mot_l_b <= '0';
234             mot_r_a <= '0'; mot_r_b <= '0';
235
236             -- El que va a la izquierda ahora va al frente
237             elsif luz_left = '1' then
238                 mot_l_a <= '1'; mot_l_b <= '0';
239                 mot_r_a <= '1'; mot_r_b <= '0';
240
241             -- El que va en reversa ahora va a la izquierda
242             elsif luz_back = '1' then
243                 mot_l_a <= '0'; mot_l_b <= '1';
244                 mot_r_a <= '1'; mot_r_b <= '0';
245
246             -- El que va al frente ahora gira a la derecha
247             elsif luz_front = '1' then
248                 mot_l_a <= '1'; mot_l_b <= '0';
249                 mot_r_a <= '0'; mot_r_b <= '1';
250
251             -- El de la derecha se queda igual
252             elsif luz_right = '1' then
253                 mot_l_a <= '0'; mot_l_b <= '1';
254                 mot_r_a <= '0'; mot_r_b <= '1';
255
256             else
257                 estado <= ESTADO_BUSCAR_LUZ;
258                 mot_l_a <= '0';
259                 mot_l_b <= '0';
260                 mot_r_a <= '0';
261                 mot_r_b <= '0';
262             end if;
263
264     -- =====
265     -- ESQUIVE:

```

```

265 -- Inicia desde que ultrasonico detecta obstaculo.
266 -- Termina cuando se cumple ESQUIRE_TIME_MS.
267 -- =====
268 when ESTADO_ESQUIRE =>
269     leds_state <= "011";
270
271     -- Solo motor izquierdo activo hacia adelante.
272     -- Motor derecho apagado.
273     mot_l_a <= '1';
274     mot_l_b <= '0';
275     mot_r_a <= '0';
276     mot_r_b <= '0';
277
278     if esquire_cnt >= ESQUIRE_MAX - 1 then
279         esquire_cnt <= 0;
280
281         if luz_front = '1' or luz_left = '1' or
282            luz_right = '1' or luz_back = '1' then
283             estado <= ESTADO_SEGUIR_LUZ;
284         else
285             estado <= ESTADO_BUSCAR_LUZ;
286         end if;
287     else
288         esquire_cnt <= esquire_cnt + 1;
289     end if;
290
291 -- =====
292 -- META: parado
293 -- =====
294 when ESTADO_META =>
295     leds_state <= "111";
296     esquire_cnt <= 0;
297
298     mot_l_a <= '0'; mot_l_b <= '0';
299     mot_r_a <= '0'; mot_r_b <= '0';
300
301     if luz_front = '0' and luz_left = '0' and
302        luz_right = '0' then
303         estado <= ESTADO_BUSCAR_LUZ;
304     end if;
305
306 when others =>
307     estado <= ESTADO_INICIO;
308     esquire_cnt <= 0;
309     mot_l_a <= '0';
310     mot_l_b <= '0';
311     mot_r_a <= '0';
312     mot_r_b <= '0';
313     leds_state <= "000";
314
315 end case;
316 end if;
317 end if;
end process;

```

```
318  
319 end Behavioral;
```

Listing 2: Código de la FSM principal fsm.vhd

19.3. Código del módulo ultrasónico HC-SR04

```
1  -- =====
2  -- ULTRASONICO HC-SR04
3  -- FPGA: Intel DE10-Lite (MAX10, 50 MHz por defecto)
4  -- Reset: activo bajo
5  --
6  -- Funcion:
7  --   Genera trigger de 10 us.
8  --   Mide el ancho del pulso ECHO.
9  --   Activa obstaculo_20cm = '1' si la distancia es <= umbral.
10 --
11 -- Formula HC-SR04 aproximada:
12 --   distancia_cm = tiempo_echo_us / 58
13 --   Para 20 cm: tiempo_echo_us = 20 * 58 = 1160 us
14 -- =====
15 library IEEE;
16 use IEEE.STD_LOGIC_1164.ALL;
17 use IEEE.NUMERIC_STD.ALL;
18
19 entity ultrasonico_hcsr04 is
20     generic (
21         CLK_FREQ_HZ      : positive := 50_000_000;
22         DIST_THRESHOLD_CM : positive := 20;
23         MEASURE_PERIOD_MS : positive := 60;
24         ECHO_TIMEOUT_MS   : positive := 30
25     );
26     Port (
27         clk          : in  STD_LOGIC;
28         reset        : in  STD_LOGIC;  -- activo bajo
29
30         echo         : in  STD_LOGIC;
31         trigger      : out STD_LOGIC;
32
33         obstaculo_20cm : out STD_LOGIC;
34         medida_lista  : out STD_LOGIC
35     );
36 end ultrasonico_hcsr04;
37
38 architecture Behavioral of ultrasonico_hcsr04 is
39
40     function us_to_ticks(us_value : positive) return natural is
41     begin
42         return (CLK_FREQ_HZ / 1_000_000) * us_value;
43     end function;
44
45     function ms_to_ticks(ms_value : positive) return natural is
46     begin
47         return (CLK_FREQ_HZ / 1_000) * ms_value;
48     end function;
49
50     constant TRIG_TICKS      : natural := us_to_ticks(10);
51     constant PERIOD_TICKS    : natural := ms_to_ticks(MEASURE_PERIOD_MS);
52
53 end;
```

```

52  constant ECHO_TIMEOUT_TICKS : natural := ms_to_ticks(ECHO_TIMEOUT_MS);
53  constant DIST_LIMIT_TICKS   : natural := us_to_ticks(58 *
54      DIST_THRESHOLD_CM);
55
56  type sensor_state_type is (
57      S_IDLE,
58      S_TRIGGER,
59      S_WAIT_ECHO_HIGH,
60      S_COUNT_ECHO
61  );
62
63  signal sensor_state : sensor_state_type := S_IDLE;
64
65  signal trigger_r      : STD_LOGIC := '0';
66  signal obstaculo_r    : STD_LOGIC := '0';
67  signal medida_lista_r : STD_LOGIC := '0';
68
69  signal period_cnt      : natural range 0 to PERIOD_TICKS      := 0;
70  signal trig_cnt       : natural range 0 to TRIG_TICKS        := 0;
71  signal timeout_cnt    : natural range 0 to ECHO_TIMEOUT_TICKS := 0;
72  signal echo_cnt       : natural range 0 to ECHO_TIMEOUT_TICKS := 0;
73
74  -- Sincronizador para ECHO, porque viene de un sensor externo.
75  signal echo_ff1       : STD_LOGIC := '0';
76  signal echo_ff2       : STD_LOGIC := '0';
77
78  begin
79
80      trigger      <= trigger_r;
81      obstaculo_20cm <= obstaculo_r;
82      medida_lista <= medida_lista_r;
83
84      -- Sincronizador de ECHO
85      process(clk, reset)
86      begin
87          if reset = '0' then
88              echo_ff1 <= '0';
89              echo_ff2 <= '0';
90          elsif rising_edge(clk) then
91              echo_ff1 <= echo;
92              echo_ff2 <= echo_ff1;
93          end if;
94      end process;
95
96      -- Maquina de medicion del sensor
97      process(clk, reset)
98      begin
99          if reset = '0' then
100              sensor_state <= S_IDLE;
101              trigger_r <= '0';
102              obstaculo_r <= '0';
103              medida_lista_r <= '0';
104              period_cnt <= 0;
105              trig_cnt <= 0;

```

```

105         timeout_cnt    <= 0;
106         echo_cnt       <= 0;
107
108     elsif rising_edge(clk) then
109         -- Pulso de un ciclo cuando termina una medicion.
110         medida_lista_r <= '0';
111
112         case sensor_state is
113
114             -- Espera antes de iniciar una nueva medicion.
115             when S_IDLE =>
116                 trigger_r    <= '0';
117                 trig_cnt     <= 0;
118                 timeout_cnt  <= 0;
119                 echo_cnt     <= 0;
120
121                 if period_cnt >= PERIOD_TICKS - 1 then
122                     period_cnt <= 0;
123                     trigger_r    <= '1';
124                     sensor_state <= S_TRIGGER;
125                 else
126                     period_cnt <= period_cnt + 1;
127                 end if;
128
129             -- Pulso TRIGGER de 10 us.
130             when S_TRIGGER =>
131                 trigger_r <= '1';
132
133                 if trig_cnt >= TRIG_TICKS - 1 then
134                     trig_cnt     <= 0;
135                     trigger_r    <= '0';
136                     timeout_cnt  <= 0;
137                     sensor_state <= S_WAIT_ECHO_HIGH;
138                 else
139                     trig_cnt <= trig_cnt + 1;
140                 end if;
141
142             -- Espera el flanco de subida de ECHO.
143             when S_WAIT_ECHO_HIGH =>
144                 trigger_r <= '0';
145
146                 if echo_ff2 = '1' then
147                     echo_cnt     <= 0;
148                     timeout_cnt  <= 0;
149                     sensor_state <= S_COUNT_ECHO;
150
151                 elsif timeout_cnt >= ECHO_TIMEOUT_TICKS - 1 then
152                     -- No llego eco: se considera sin obstaculo
153                     cercano.
154                     obstaculo_r    <= '0';
155                     medida_lista_r <= '1';
156                     timeout_cnt    <= 0;
157                     sensor_state   <= S_IDLE;

```

```

158         else
159             timeout_cnt <= timeout_cnt + 1;
160         end if;
161
162         -- Cuenta cuanto tiempo ECHO permanece en alto.
163         when S_COUNT_ECHO =>
164             if echo_ff2 = '0' then
165                 if echo_cnt <= DIST_LIMIT_TICKS then
166                     obstaculo_r <= '1';
167                 else
168                     obstaculo_r <= '0';
169                 end if;
170
171                 medida_lista_r <= '1';
172                 echo_cnt <= 0;
173                 sensor_state <= S_IDLE;
174
175                 elsif echo_cnt >= ECHO_TIMEOUT_TICKS - 1 then
176                     -- Pulso demasiado largo o sensor atorado.
177                     obstaculo_r <= '0';
178                     medida_lista_r <= '1';
179                     echo_cnt <= 0;
180                     sensor_state <= S_IDLE;
181
182                 else
183                     echo_cnt <= echo_cnt + 1;
184                 end if;
185
186             end case;
187         end if;
188     end process;
189
190 end Behavioral;

```

Listing 3: Código del módulo ultrasónico ultrasonico_hcsr04.vhd